

Processors and I/O

EMBEDDED — Lecture 4

Processor and Input and Output Devices

Peripherals and Chip Architecture

Other than the CPU, a chip integrates many peripherals (e.g., I/O devices): timers, UART, sensors.

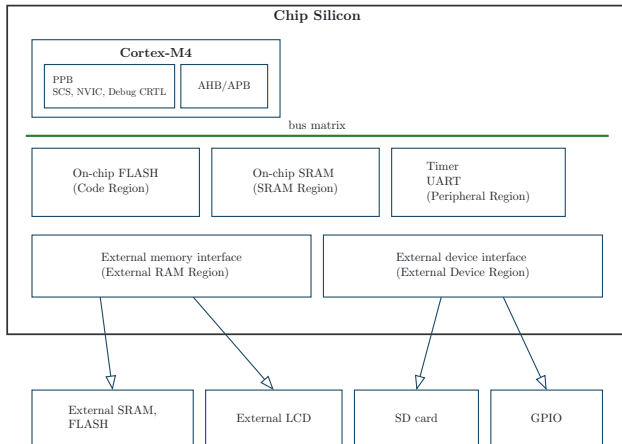


Fig. 1: MCU architecture: Cortex-M4 core, on-chip memories, peripherals and external interfaces.

How Do They Communicate?

- The processor reaches on-chip peripherals through the **bus matrix**; external interfaces connect off-chip devices.
- Each peripheral (i.e., device) exposes registers; the CPU talks to the device by reading and writing device registers.
- Interrupt lines (not shown in this figure) will report events from I/O devices that need CPU service.

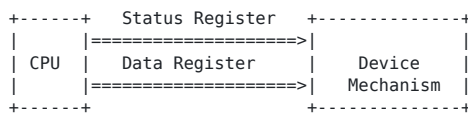


Fig. 2: Status and data registers between CPU and device.

Device registers connect software to hardware state:

Data registers

- carry values moving into or out of the device;
- hold data values;
- access direction depends on the peripheral;
- can be readable or writable.

Status and control registers

- report events and configure operating modes;
- e.g., event: the current operation has been completed;
- individual bits often have different meanings.

Two Architectural Models for Device Access

Two ways to access those registers:

- I/O instructions:** special instructions for input and output (in and out in the case of the Intel x86); provide a separate address space for I/O devices (separate address lines...).
- Memory-mapped I/O:** provides addresses for the registers in each I/O device; programs use the CPU's normal memory read and write instructions to communicate with the devices.

Model	Addressing	Instructions	Typical example
Separate I/O space	Dedicated device address space	Special input and output instructions	Legacy ports x86
Memory-mapped I/O	Device registers share the memory address space	Normal load and store instructions	Arm microcontrollers

ARM Loads and Stores Access Device Registers

```
DEV1_BASE EQU 0x40001000
LDR r1, = DEV1_BASE
LDR r0, [r1]           // read register
```

```
MOVS r0, #8
STR r0, [r1]           // write register
```

What matters A wrong address can control a different peripheral or trigger a bus fault. The address must identify the correct register. The access width and alignment must match the peripheral specification.

C Requires Volatile Access to Device Registers

We can use pointers to manipulate the addresses of I/O devices.

```
#include <stdint.h>
#define DEV1_STATUS_ADDR 0x40001000u
#define DEV1_STATUS (*(volatile uint32_t *)DEV1_STATUS_ADDR)
uint32_t status = DEV1_STATUS;
DEV1_STATUS = 8u;
```

volatile Tells the compiler that every read and write matters because hardware may change the value outside normal program flow.

Introduce functions that read and write memory locations:

```
#define DEV1_STATUS_ADDR 0x40001000u
int read(uint8_t *location) {
    return *location;
}
void write(uint8_t *location, char newval) {
    (*location) = newval;
}
uint32_t status = read(DEV1_STATUS_ADDR); /* read device register */
write(DEV1_STATUS_ADDR, 8); /* write 8 to device register */
```

Register Definitions Should Express Width and Intent

The device reference manual defines the legal access pattern for each device register.

Concern	Reason
Access width	A byte, halfword, or word transfer may have different hardware meaning
Read-only and write-only behavior	Some registers reject unsupported access directions
Reserved bits	Software should preserve or write required values
Side effects	A read may clear status and a write may acknowledge an event

Polling (Busy-Wait) I/O

- I/O devices are typically slower than the CPU and may require many cycles to complete an operation.
- Busy-wait** (often called polling): asking an I/O device whether it is finished by reading its status register; software reads a status register until the ready or complete condition appears.
- The loop provides low control-flow complexity; CPU cycles are consumed while no useful application work progresses.

Polling an Output Device (Example)

One character starts each transaction; the inner loop waits for completion.

```
#define OUT_CHAR 0x1000 /* output device character register */
#define OUT_STATUS 0x1001 /* output device status register */
while (*current != '\0') {
    OUT_DATA = *current;
    OUT_STATUS = START;
    while (OUT_STATUS != DONE) {
        // CPU waits here
    }
    current++;
}
```

The CPU cannot perform independent application work during the wait.

A Polling-Based Input-to-Output Loop (Example)

Repeat forever:

- wait until the input status reports available data;
- read one value from the input data register;
- write that value into the output data register;
- wait until the output device completes the transaction.

```
for (;;) {
    while (IN_STATUS == EMPTY) { }
    uint8_t c = IN_DATA;
    while (OUT_STATUS == BUSY) { }
```

```
OUT_DATA = c;
OUT_STATUS = START;
}
```

The loop does not allow the foreground work to continue.

Interrupt-driven I/O

- Busy-wait I/O is inefficient; the CPU could do useful work in parallel with the I/O.
- The interrupt mechanism allows a device to request service only when an event occurs.
- The processor saves context, runs a handler, then resumes the interrupted program.

Several Signals Control the Interrupt Process

- The device asserts an **interrupt request**; the I/O device's logic decides when to interrupt (e.g., when data is ready, the user pressed a button).
- The CPU accepts the request: it asserts the **interrupt acknowledge** signal and changes the program counter to point to the device's interrupt handler.
- The interrupt handler identifies the event source (e.g., which device) and takes the necessary processing actions.

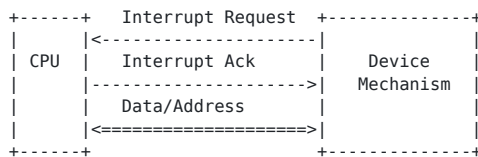


Fig. 3: Signals between CPU and device: request, acknowledge, data/address.

Interrupt Entry Changes Control Flow

When an interrupt occurs:

- first, the value of the PC at the interruption is saved: the CPU can return to the foreground program later;
- then, the PC starts pointing to an interrupt handler routine: this routine takes care of the device by reading data that have just become ready, writing the next data...

Phase	Processor action
Recognition	Detect interrupt
Context save	Preserve the return state and selected registers
Vector fetch	Load the handler address from the vector table
Service	Execute the interrupt handler
Return	Restore state and resume the interrupted code

Cortex-M Interrupt Entry

- Save the necessary context: pushes important registers R0 to R3, R12, LR, PC, and xPSR into stack.
- Finds the handler address from the interrupt vector table.
- The handler may save additional registers that it uses.
- Interrupt return restores the hardware-saved state and context.
- Interrupt code in C is processed in a specific way by the compiler: compiler-generated handlers follow the architecture calling convention when declared correctly.

A Minimal Input Interrupt Handler in C

- Read the data that caused the interrupt.
- Record minimal state for later processing.
- Acknowledge or clear the device event.
- Return quickly.

```
volatile uint8_t latest_char;
volatile bool char_ready;
void input_handler(void) {
    latest_char = IN_DATA; // read device register
    char_ready = true;
    IN_STATUS = ACK; // acknowledge the device
}
```

Interrupts (Example)

achar passes the character to the foreground program; gotchar signals when a new character has been received.

```

/* get a character and put in global (called when IN_STATUS is
1) */
void input_handler() {
    achar = read(IN_DATA); /* get character */
    gotchar = TRUE; /* signal to main program */
    write(IN_STATUS, 0); /* reset status to initiate next transfer
*/
}
/* react to character being sent (called when OUT_STATUS is 0)
*/
void output_handler() {
    /* don't have to do anything */
}

```

The main program is somewhat simpler compared to Busy Wait I/O, but still does not let the foreground program do useful work: it polls input and writes data.

```
main() {
    while (TRUE) { /* read then write forever */
        if (gotchar) { /* write a character */
            write(OUT_DATA, achar); /* put character in device */
            write(OUT_STATUS, 1); /* set status to initiate write */
            gotchar = FALSE; /* reset flag */
        }
    }
}
```

Circular Buffer

A Circular Buffer Separates Producer and Consumer Timing

- One index identifies the next read position.
- The other identifies the next write position.

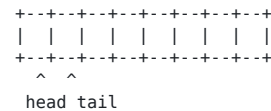


Fig. 4: Empty circular buffer: head and tail coincide.

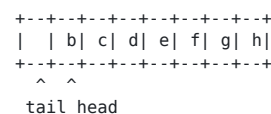


Fig. 5: Circular buffer holding data b–h.

Circular Buffer Implementation

```
#define BUF_SIZE 8
char io_buf[BUF_SIZE]; /* character buffer */
int buf_head = 0, buf_tail = 0; /* current position in buffer */
int error = 0; /* set to 1 if buffer ever overflows */

int buffer_empty() { /* returns TRUE if buffer is empty */
    return buf_head == buf_tail;
}

int buffer_full() { /* returns TRUE if buffer is full */
    return (buf_tail+1) % BUF_SIZE == buf_head ;
}

int nchars() { /* returns the number of characters in the buffer */
    if (buf_head >= buf_tail)
        return buf_head - buf_tail;
    else
        return BUF_SIZE - buf_tail - buf_head;
}

void buffer_put(char achar) { /* add a character to the buffer head */
    io_buf[buf_tail++] = achar;
    /* check pointer */
    if (buf_tail == BUF_SIZE)
        buf_tail = 0;
}

char buffer_get() { /* take a character from the buffer head */
    char achar;
    achar = io_buf[buf_head++];
    /* check pointer */
    if (buf_head == BUF_SIZE)
        buf_head = 0;
    return achar;
}
```

Input ISR Adds Data and Starts Output

The input ISR adds data and starts output:

- capture the arriving item;
- record overflow without blocking;
- kick the output only when the queue was idle;
- acknowledge the input event.

```
void input_handler(void) {
    uint8_t c = IN_DATA;
    if (buffer_full()) {
        overflow = true;
    } else {
        bool was_empty = buffer_empty();
        buffer_put(c);
        if (was_empty) start_output();
    }
    IN_STATUS = ACK;
}
```

Each completion interrupt launches the next queued item; the handler disables unnecessary interrupts when no data remains.

```
void output_handler(void) {
    OUT_STATUS = ACK;
    if (!buffer empty()) {
```

```

    OUT_DATA = buffer_get();
    OUT_CONTROL = START;
} else {
    stop_output();
}
}

```

Foreground/ISR Timing

The foreground program is occasionally interrupted by input and output operations, handled by the interrupt handlers in the background; the foreground program resumes after interrupt handlers.

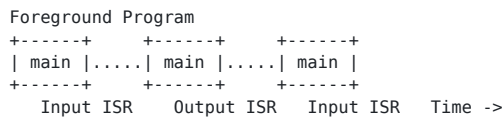


Fig. 6: Periodic interruptions of the foreground by the ISRs.

```

void main(){
    ...
    for (i = 0; i < M; i++) {
        y[i] = b[i];
        for (j = 0; j < N; j++)
            y[i] = y[i] + A[i,j]*x[j];
    }
    ...
}

```

Bugs

The errors can be very hard to find when the interrupt handlers are buggy due to the concurrency.

Saving Registers

An interrupt handler:

- must save any CPU register that it will modify;
- must restore them before it exits.

Forgetting to save/restore a register in the handler might cause that register to mysteriously change a variable in the foreground program.

Read-Modify-Write

For example, in order to set bit[3] in word data in address 0x20000000:

```

...
LDR R1, =0x20000000 ;Setup address
STR R2, [R1]         ;Write back 0x79
...

```

```

;Read-Modify-Write Operation
LDR R1, =0x20000000 ;Setup address
LDR R0, [R1]         ;Read 0x21
ORR.W R0, #0x8       ;Set bit[3]
STR R0, [R1]         ;Write back 0x29

```

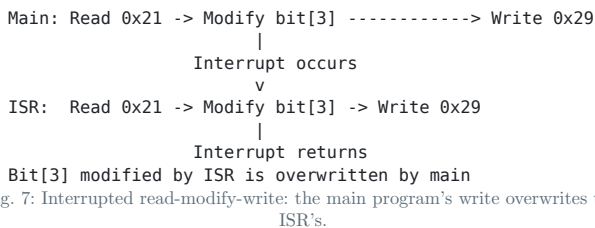


Fig. 7: Interrupted read-modify-write: the main program's write overwrites the ISR's.

Read-modify-write operation:

- reads the data (0x21) from the address 0x20000000;
- the interrupt changes the data of 0x20000000 address;
- writes back the modified old data back;
- 0x79 has been lost!

Bit[3] modified by ISR is overwritten by the main program.

Interrupt Handlers Should Do Bounded Work

- Capture or deliver the minimum data required by the device.
- Clear the interrupt source before it can retrigger unexpectedly.
- Avoid blocking calls and unbounded loops.
- Move expensive processing into foreground or deferred work.
- Measure worst-case execution time.

Interrupts — Implementation

- The CPU checks the interrupt request (IRQ) line at every instruction.
- If an interrupt request has been asserted, the CPU:
 - puts the return address on a stack (as CPU does for subroutines);
 - does not fetch the instruction pointed to by the PC;
 - sets the PC to the beginning of the interrupt handler.
- The address of the interrupt handlers are stored in a table.

Priorities and Vectors

- Most systems have more than one I/O device: multiple devices can interrupt, several interrupt handlers and device addresses.
- **Interrupt priorities:** recognize some interrupts as more important than others.
- **Interrupt vectors:** allow the interrupting device to specify its interrupt handler.

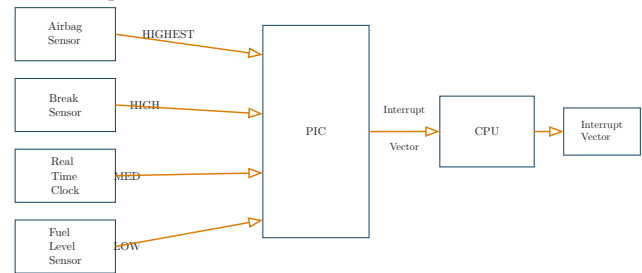


Fig. 8: Source priorities: the PIC selects the highest-priority request and presents the vector to the CPU.

Programmable Interrupt Controller (PIC)

The PIC prioritizes multiple interrupt sources so that at any time the highest priority interrupt is presented to the core CPU for processing. Cortex-M integrates this function in the NVIC.

Nested Interrupts

- Priority logic selects the highest eligible request.
- The PIC stores the priority level of that interrupt in an internal register.
- When a subsequent interrupt occurs, the priority is checked against current priority.
- **Nested interrupts:** a higher priority interrupt source can preempt the processing of a lower priority interrupt.

Pending Bits and Masking

- **Pending bits** record interrupts awaiting service.
- **Masking:** enable bits determine which sources may interrupt.
- The highest-priority interrupt is called the **nonmaskable interrupt** (NMI); the NMI cannot be turned off.
- E.g., usually reserved for interrupts caused by power failures to save critical state in nonvolatile memory, turn off I/O devices...

Interrupt Vector Table

- Each interrupt has a **vector number**.
- The vector number indexes the vector table.
- The selected entry holds the **interrupt vector:** i.e., the memory address of the interrupt handler.

```

void handler1() {
    ...
}

```

Vector	Handler
Vector 0	Handler 1
Vector 1	Handler 3
Vector 2	Handler 4
Vector 3	Handler 2

Overhead of Interrupts

- An interrupt causes a change in the program counter: incurs a branch penalty.
- Interrupt might automatically store some CPU registers: requires extra cycles.
- Acknowledge the interrupt and obtain the interrupt vector: requires extra cycles.
- The interrupt handler overhead: saves and restores CPU registers that were not automatically saved; the interrupt return instruction restores the automatically saved state, incurs a branch penalty.

Interrupt Response Time

The time required for the hardware to respond to the interrupt, obtain the vector, save state and so on cannot be changed by the programmer.

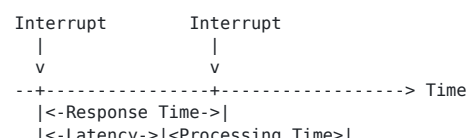


Fig. 9: Interrupt latency, response time and processing time.

Time between interrupts Interval between successive interrupts, relative to latency, response time and processing time.